

用 Kalman Control 增强 Flow-Based 图像编辑一致性

Haozhe Chi Zhicheng Sun Yang Jin Yi Ma Jing Wang Yadong MU

中文技术阅读版: OpenReview ExVMnClnrM, NeurIPS 2025 poster

Abstract

这篇论文提出 Kalman-Edit, 用控制论和 Kalman filter 改善 flow-based image editing 的结构一致性。核心问题是: 指令编辑应只改变目标区域, 但现有 rectified-flow 编辑方法经常让背景、人物身份或局部细节漂移。作者把漂移解释为多步生成中的误差累积, 并把编辑过程改写为带噪声的 Linear Quadratic Gaussian (LQG) 控制问题。算法先用 LQR 控制得到混合语义 latent, 再用原图与中间图的历史 inversion 轨迹作为测量序列, 在生成后半段通过 Kalman filter 把结构细节补回去。业务收益是更高的非目标区域一致性, 同时保持可编辑性; 代价是标准版需要第二次 inversion, 加速版 Kalman-Edit* 用 shortcut 换取速度但会牺牲部分低层结构信息。

1 业务问题: 编辑质量和结构一致性的冲突

Flow/rectified-flow 模型可以把图像编辑写成从源图 latent 到目标 prompt latent 的轨迹控制问题。对用户而言, 真正重要的不是“图像变了”, 而是“只该变的地方变了”。例如给人脸加眼镜时, 身份、发型和脸型不能漂移; 给街景加物体时, 道路、建筑和背景布局不能乱。

论文指出, 已有 controller-based editing 有一个典型困境: 控制太强会 under-edit, 图像仍像原图; 控制太弱会 over-edit, 目标 prompt 达到了但结构漂移; 中等控制又可能产生语义含混。Kalman-Edit 的商业价值, 就是在可编辑性和结构一致性之间提供一个更稳定的折中。

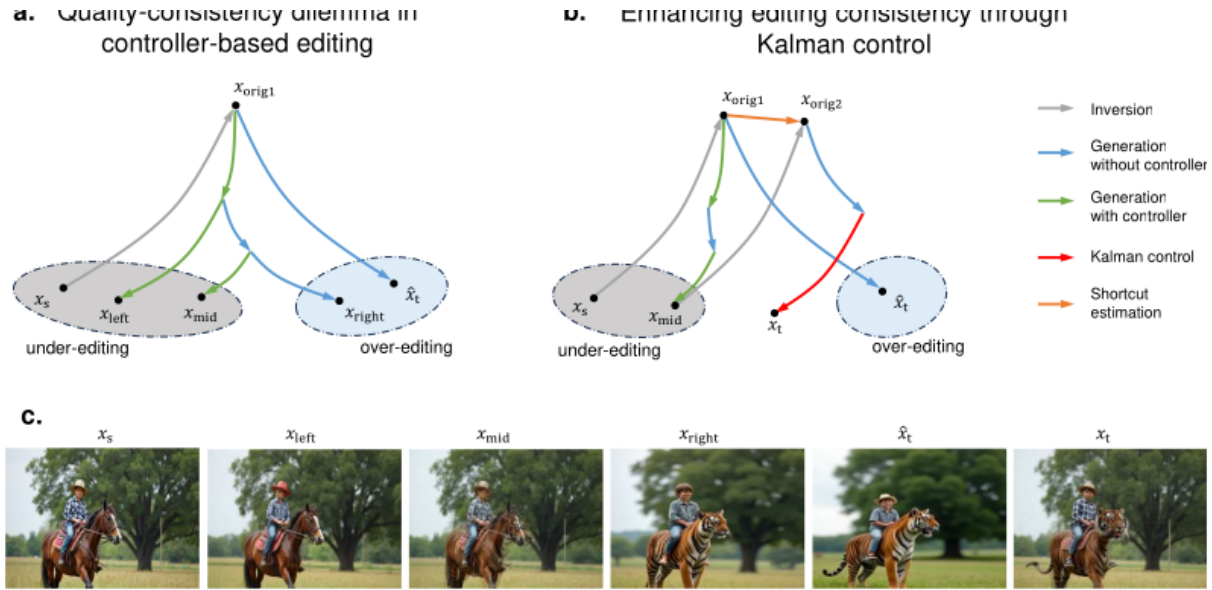


Figure 1: Conceptual illustration of a previously-developed controller-based method [43] (left) and our method (right). x_s represents original image, while $\hat{x}_t$ and x_t represent edited images. The controller-based method derives an optimal control strategy for different stages of the generation process. However, as illustrated in the figure for $\alpha = 1$, excessive control leads to failed edits, whereas $\alpha = 0$ leads to structural drift.

Figure 1: 原论文 Figure 1: 传统 controller-based editing 的质量—一致性困境, 以及 Kalman control 如何利用历史轨迹修正生成路径。

2 Rectified Flow 编辑与 LQR 控制

Flow-based generative model 用 ODE 表示从源分布到目标分布的概率路径:

$$\frac{dx_t}{dt} = V(x_t, t). \quad (1)$$

Rectified flow 使用线性插值

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad (2)$$

因此通常比传统 diffusion sampling 更直接。给定源图 $\mathbf{x}_s$ 、源 prompt c_s 、目标 prompt c_t ，基础编辑流程是：

$$\text{Inversion : } \frac{d\mathbf{x}_t^s}{dt} = V(\mathbf{x}_t^s, t, c_s), \quad (3)$$

$$\text{Generation : } \frac{d\mathbf{x}_t^t}{dt} = V(\mathbf{x}_t^t, t, c_t). \quad (4)$$

直接这样做容易偏离原图结构。前作将其写成最优控制：

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t, \quad (5)$$

并最小化二次成本

$$J_1 = \mathbf{x}_1^\top \mathbf{F} \mathbf{x}_1 + \int_0^1 (\mathbf{x}_t^\top \mathbf{Q} \mathbf{x}_t + \mathbf{u}_t^\top \mathbf{R} \mathbf{u}_t) dt. \quad (6)$$

在 rectified flow 设定下，可得到类似 LQR 的控制器：

$$\mathbf{u}_t = \frac{\mathbf{x}_s - \mathbf{x}_t}{1 - t}. \quad (7)$$

这个控制器会把轨迹拉回原图，但单独使用仍无法稳定保存背景细节，因为每一步的小误差会向后传播。

3 Kalman-Edit 的核心思想

3.1 把编辑改写成 LQG 问题

Kalman-Edit 的关键转向是：把每一步 velocity update 看成带噪声的控制过程，而不是完全确定的轨迹。论文写成

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{y}_t = \mathbf{H}\mathbf{x}_t + \boldsymbol{\sigma}_t. \quad (8)$$

其中 $\mathbf{y}_t$ 是测量序列，来自 inversion 历史轨迹； $\mathbf{w}_t$ 和 $\boldsymbol{\sigma}_t$ 是状态与测量噪声。Kalman filter 用以下递推估计更可信的状态：

$$\mathbf{K}_k = \mathbf{P}_{k-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{k-1} \mathbf{H}^\top + \mathbf{T})^{-1}, \quad (9)$$

$$\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{K}_k(\mathbf{y}_k - \mathbf{H}\mathbf{x}_{k-1}), \quad (10)$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_{k-1}. \quad (11)$$

直观上， $\mathbf{K}_k(\mathbf{y}_k - \mathbf{H}\mathbf{x}_{k-1})$ 是“看历史轨迹后发现当前生成哪里偏了”的修正项。它不是简单把原图粘回来，而是在生成后半段用历史 inversion latent 作为观测来纠偏。

3.2 两阶段设计

论文没有直接在一次 inversion/generation 里硬套 Kalman filter，因为这会把轨迹拉向原图和目标图之间的劣质中间态。Kalman-Edit 因此分两阶段：

1. 第一阶段：从原图 inversion 得到 $\mathbf{x}_{\text{orig}1}$ ，用 LQR controller 生成中间图 $\mathbf{x}_{\text{mid}}$ 。这个中间图同时含有源图结构和目标 prompt 语义。
2. 第二阶段：标准版再次 inversion $\mathbf{x}_{\text{mid}}$ 得到 $\mathbf{x}_{\text{orig}2}$ ，再用 Kalman filter 生成最终图 $\mathbf{x}_t$ 。

测量序列把原图和中间图的 inversion 历史拼起来：

$$\mathbf{y}_k = \text{Inv}(\mathbf{x}_s, k), \quad 1 \leq k \leq \delta - 1, \quad \mathbf{y}_k = \text{Inv}(\mathbf{x}_{\text{mid}}, k), \quad \delta \leq k \leq l. \quad (12)$$

这相当于前段保留原图结构，后段吸收目标语义。实际 flow 模型通常预测 velocity，因此论文实现时更新的是速度：

$$\mathbf{x}_{t_{k+1}}^t = \mathbf{x}_{t_k}^t + (t_{k+1} - t_k) \mathbf{v}_{t_k}', \quad (13)$$

$$\mathbf{v}_{t_k}' = \mu \mathbf{v}_{t_k} + \lambda \mathbf{u}_{t_k} + (1 - \mu - \lambda) \mathbf{K}_k(\mathbf{y}_k - \mathbf{H}\mathbf{x}_{t_k}). \quad (14)$$

算法 1: Kalman-Edit / Kalman-Edit* 中文化流程

输入: 原图 $\mathbf{x}_s$; 总步数 N ; 源 prompt c_s ; 目标 prompt c_t ; 反演函数 $\text{Inv}(\cdot, \cdot)$; 测量长度 l ; 切换点 δ
 初始化: 设置 LQR controller、Kalman 初始协方差 $\mathbf{P}_0$ 、观测矩阵 $\mathbf{H}$ 和噪声矩阵 $\mathbf{T}$

```

1  $\mathbf{x}_{\text{orig1}} \leftarrow \text{Inv}(\mathbf{x}_s, N)$ 
2 使用 LQR controller 从  $\mathbf{x}_{\text{orig1}}$  生成中间 latent / 图像  $\mathbf{x}_{\text{mid}}$ 
3 若 使用标准 Kalman-Edit 则
4    $\mathbf{x}_{\text{orig2}} \leftarrow \text{Inv}(\mathbf{x}_{\text{mid}}, N)$ 
5 否则
6    $\mathbf{x}_{\text{orig2}} \leftarrow \mathbf{x}_{\text{orig1}} + \mathbf{x}_{\text{mid}} - \mathbf{x}_s$  // shortcut
7 对  $k = 1, \dots, l$  执行
8   若  $k < \delta$  则
9      $\mathbf{y}_k \leftarrow \text{Inv}(\mathbf{x}_s, k)$ 
10  否则
11     $\mathbf{y}_k \leftarrow \text{Inv}(\mathbf{x}_{\text{mid}}, k)$ 
12 对 生成后半段 step  $k$  执行
13   计算  $\mathbf{K}_k = \mathbf{P}_{k-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{k-1} \mathbf{H}^\top + \mathbf{T})^{-1}$ 
14   用  $\mathbf{v}'_{t_k} = \mu \mathbf{v}_{t_k} + \lambda \mathbf{u}_{t_k} + (1 - \mu - \lambda) \mathbf{K}_k (\mathbf{y}_k - \mathbf{H} \mathbf{x}_{t_k})$  更新 velocity
15   更新  $\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_{k-1}$ 

```

输出: 最终编辑结果 $\mathbf{x}_t$

Table 1: SFHQ 人脸编辑结果。Kalman-Edit 在身份和结构保持上最稳。

方法	Face Rec.↓	CLIP-I↑	LPIPS↓	CLIP-T↑	DreamSim↓
RF-Edit	0.4051	0.8984	0.1562	0.2910	0.1591
RF-Inversion	0.4325	0.8927	0.1720	0.3012	0.1889
FlowEdit	0.4856	0.8579	0.1687	0.2905	0.2375
FlowChef	0.4013	0.8769	0.1401	0.2832	0.1487
Kalman-Edit	0.3958	0.9167	0.1332	0.2921	0.1408
Kalman-Edit*	0.4696	0.8871	0.1892	0.2936	0.2227

3.3 Kalman-Edit* 的 shortcut

标准版需要对 $\mathbf{x}_{\text{mid}}$ 做第二次 inversion，成本较高。加速版用一阶近似：

$$\mathbf{x}_{\text{orig2}} = \mathbf{x}_{\text{orig1}} + (\mathbf{x}_{\text{mid}} - \mathbf{x}_s). \quad (15)$$

这个式子可理解为“把第一次 inversion 的起点沿编辑差分平移”。它省掉第二次 inversion，所以更快；但当目标编辑幅度较大或低层纹理变化明显时，会损失一部分结构细节。

4 实验结论

4.1 定量表现

论文在 SFHQ、HQ、ZONE 和 DIV2K 上评估。指标含义可以分成两类：CLIP-T 衡量 prompt adherence；Face Rec.、CLIP-I、DINO、LPIPS、DreamSim 衡量不同层级的一致性。下面只摘录主文中最能说明业务收益的结果。

4.2 定性表现

从可视化看，Kalman-Edit 的优势集中在“保留原图非目标区域”。例如 ZONE/DIV2K 案例中，baseline 往往能加上目标对象，却会丢道路锥、桌面纹理、植物局部或窗外结构；Kalman-Edit 更倾向于把这些细节留下来。人脸案例中，它在加眼镜或胡子时更能保留身份、脸型 and 发型。

Table 2: ZONE/DIV2K 结果。标准版在结构保持类指标上优势明显，加速版偏向编辑语义。

方法	CLIP-T $\uparrow$	CLIP-I $\uparrow$	LPIPS $\downarrow$	DINO $\uparrow$	DreamSim $\downarrow$
RF-Edit	0.2964	0.8926	0.2039	0.7986	0.1776
RF-Inversion	0.2844	0.8919	0.2491	0.7974	0.1536
FlowEdit	0.3096	0.8687	0.2269	0.7671	0.2319
FlowChef	0.3025	0.8831	0.2563	0.7456	0.2471
Kalman-Edit	0.2957	0.9492	0.1407	0.9141	0.0793
Kalman-Edit*	0.3220	0.8986	0.2488	0.8237	0.1454



Figure 2: 原论文 Figure 2: ZONE/DIV2K 上的结构保持与编辑质量对比。



Figure 3: Qualitative results obtained using human face images from SFHQ dataset. The first two rows are edited with target prompt “A person wearing glasses” and the last row is edited with target prompt “A person with beard”. The edited results are compared with baseline methods, demonstrating our approach’s superior ability to preserve the structural details of human faces (*i.e.*, our method produces edited images of higher fidelity, recovering facial features more accurately than baseline

Figure 3: 原论文 Figure 3: SFHQ 人脸编辑结果。Kalman-Edit 在加眼镜、加胡子时更能保留身份细节。

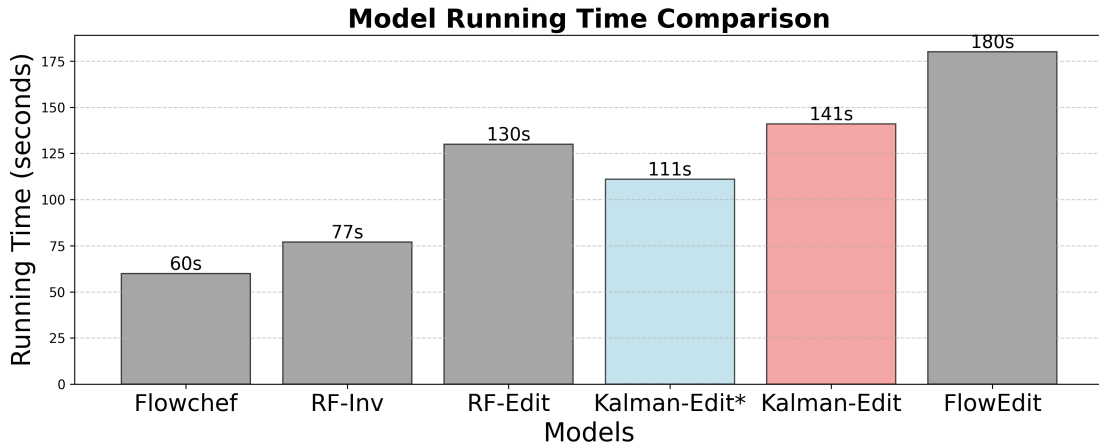


Figure 4: 原论文 Figure 4: 运行时间对比。Kalman-Edit* 比标准版更快，但仍慢于 FlowChef 和 RF-Inversion。

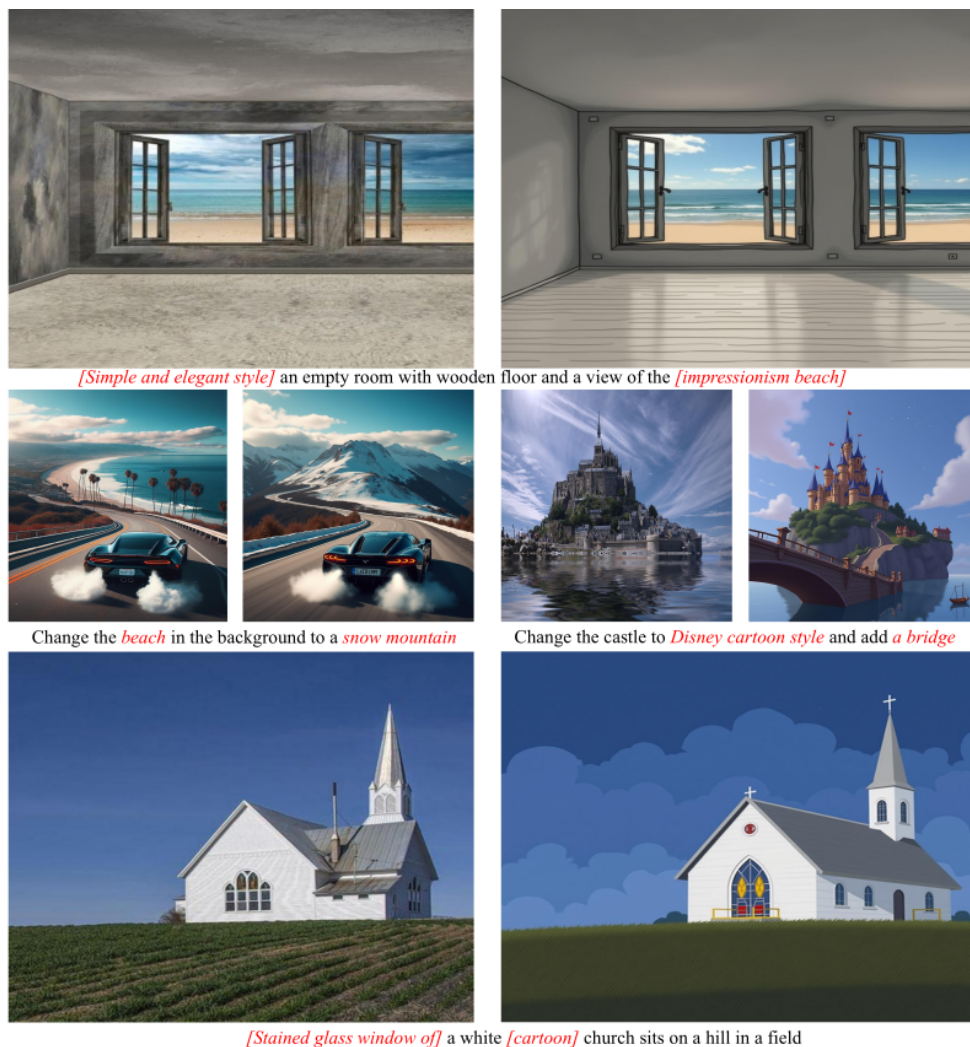


Figure 5: High-resolution qualitative results of style transfer and scene editing tasks. The left image is the original input and the right one is the edited result. As illustrated above, our method achieves precise prompt adherence and delivers high-quality editing outcomes.

Figure 5: 原论文 Figure 5: 风格迁移和场景编辑示例，展示复杂 prompt 下的结构保持能力。

5 如何理解这篇工作的贡献

从工程角度看，Kalman-Edit 的创新不是“把 Kalman filter 生硬套到图片上”，而是把 flow editing 的历史轨迹变成可用测量信号。原图 inversion 轨迹提供结构细节，中间图 inversion 轨迹提供目标语义，两者通过 Kalman gain 控制进入后半段生成过程。

这篇工作的优势是：训练免费；适合搭在已有 rectified-flow 编辑模型上；在非目标区域一致性上收益清晰。主要劣势是：标准版需要两次 inversion，速度成本较高；shortcut 版虽然快，但低层结构指标下降；object removal 这类“应该删除原有细节”的任务需要额外调节，否则 Kalman filter 可能把不该保留的原图残留带回来。

6 结论

Kalman-Edit 解决的是图像编辑产品中很实际的质量问题：目标区域要听指令，非目标区域要稳。论文用 LQG/Kalman control 把历史轨迹纳入生成更新，缓解了 rectified-flow 多步编辑中的误差累积。它给 flow-based editing 提供了一条有业务价值的方向：不是只追求更强生成能力，而是在可控编辑中显式管理历史、误差和结构一致性。